EE 3EY4
Lab 2

Ryan Li – li2323 – 400451015

Sungkyun Cho – chos45 - 400464608

January 19, 2026

Objective 3:

Question 1: What is the purpose of the entries /rosout and /rosout_agg listed in Figure 10? Explain in detail

/rosout: All active nodes on the Jetson Nano publish their status and error messages to this topic. The rosout node (part of roscore) subscribes to it to collect these messages.

/rosout_agg: The agg stands for aggregate. The rosout node then "aggregates" (combines) these messages. This prevents the system from having to create numerous connections just to see logs. The rosout node publishes the combined stream to /rosout_agg

Question 2: Break down the parts of the command in Figure 11 and explain each function: rostopic pub, /hello, std_msgs/String, “Hello Robot”.

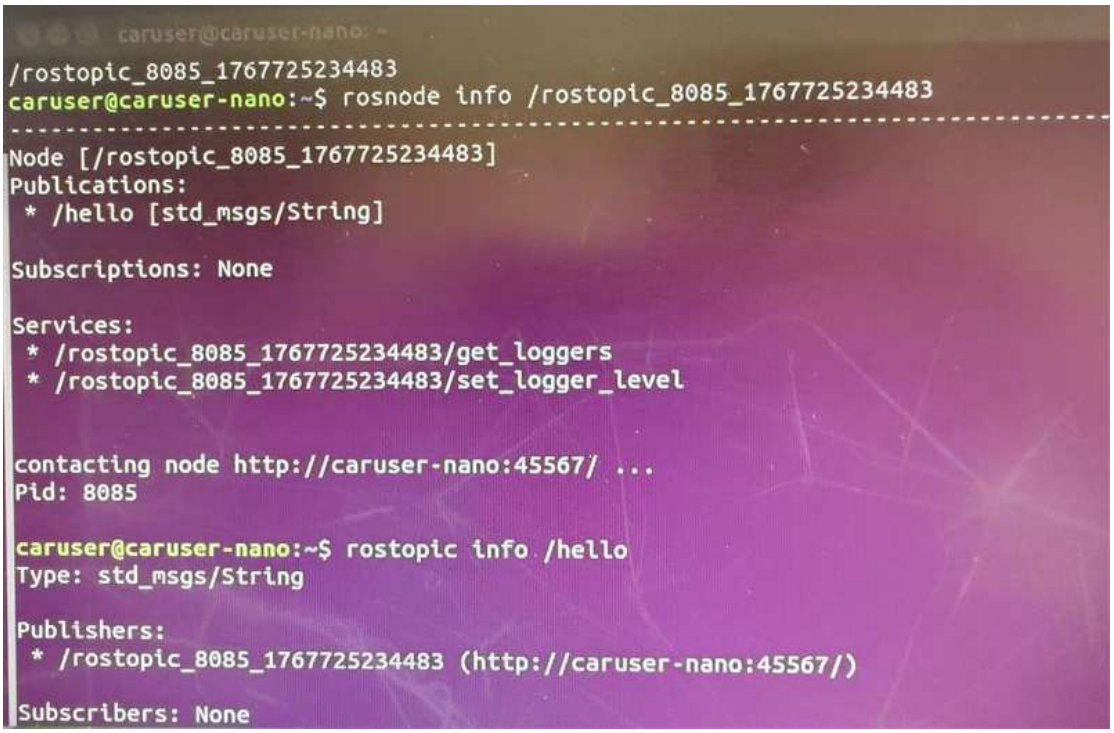
rostopic pub: This is the base command and the action. It tells ROS that you want to act as a Publisher and send data manually to a specific topic.

/hello: This is the topic name. This is the specific communication channel where the data is sent.

std_msgs/String: This defines the message type. It defines the structure of the message as a standard text string.

“Hello Robot”: This is the actual data. This is the actual data being broadcast to any subscribers.

Question 3: Write and explain in your own words a summary of what you did in this objective and what you learned from it. Include screenshots of the output of the rosnode info /rostopic/_…. command and of the rostopic info /hello command



In this objective, we explored the foundational communication architecture of the Robot Operating System (ROS) by initializing the environment and observing how data flows between processes. We began by launching roscore in the Linux terminal to start the ROS Master, which acts as the central coordinator for all nodes in the system. Using the rostopic list command, we identified the default logging topics, /rosout and /rosout_agg, which the system uses to aggregate and broadcast status messages from all active nodes.

In this objective, we explored the ROS communication framework by manually publishing a std_msgs/String message to a new topic called /hello. By using rostopic echo, we verified the data transmission and observed the Publisher/Subscriber model in real-time. Our investigation using rosnode info and rostopic info revealed that ROS automatically instantiated a background node to handle our manual broadcast. This exercise demonstrated that nodes operate as independent, anonymous processes that communicate solely through topics. Mastering these inspection tools provided us with a foundation for debugging complex robotic systems on the Jetson Nano.

Objective 4:

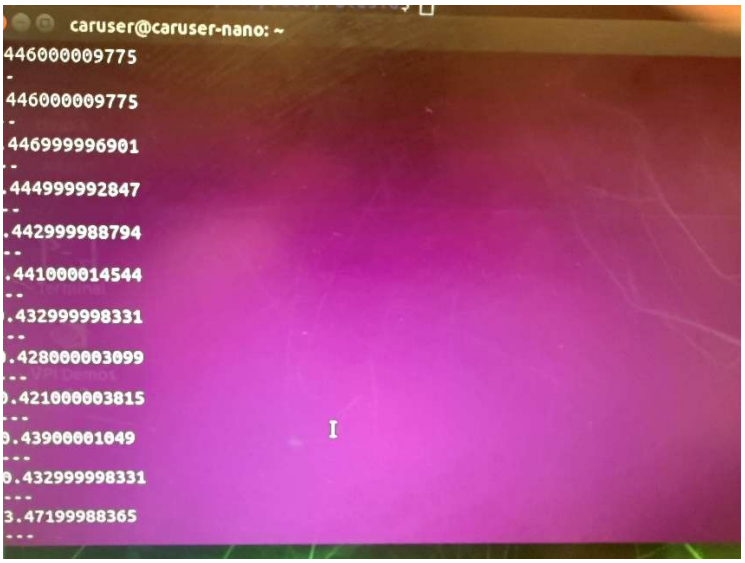
Question 4: In your report, explain why you need to apply these commands and what they mean

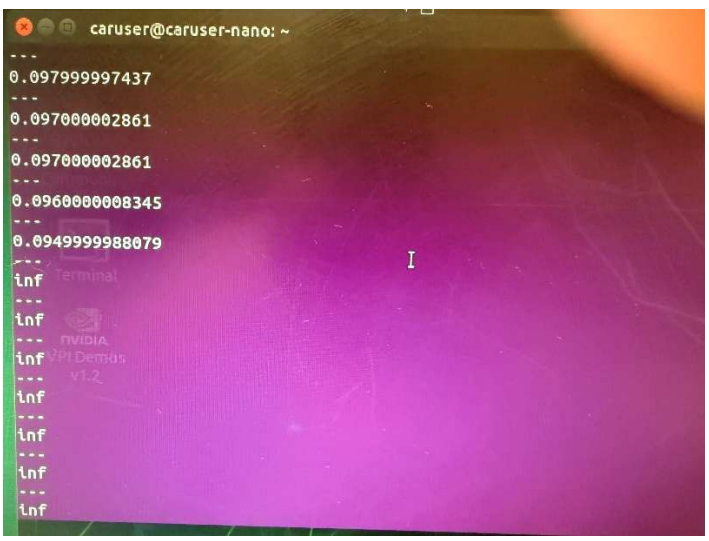
$ sudo udevadm control --reload-rules: This command tells the udev daemon to refresh its internal database by re-reading the rule files from the disk. This ensures any manual changes or new rules we added are loaded into the system's memory.

$ sudo udevadm trigger: While reloading rules updates the database, it does not automatically apply them to hardware that is already plugged in. The trigger command forces the kernel to "re-discover" connected devices and apply the newly loaded rules to them immediately.

Objective 5:

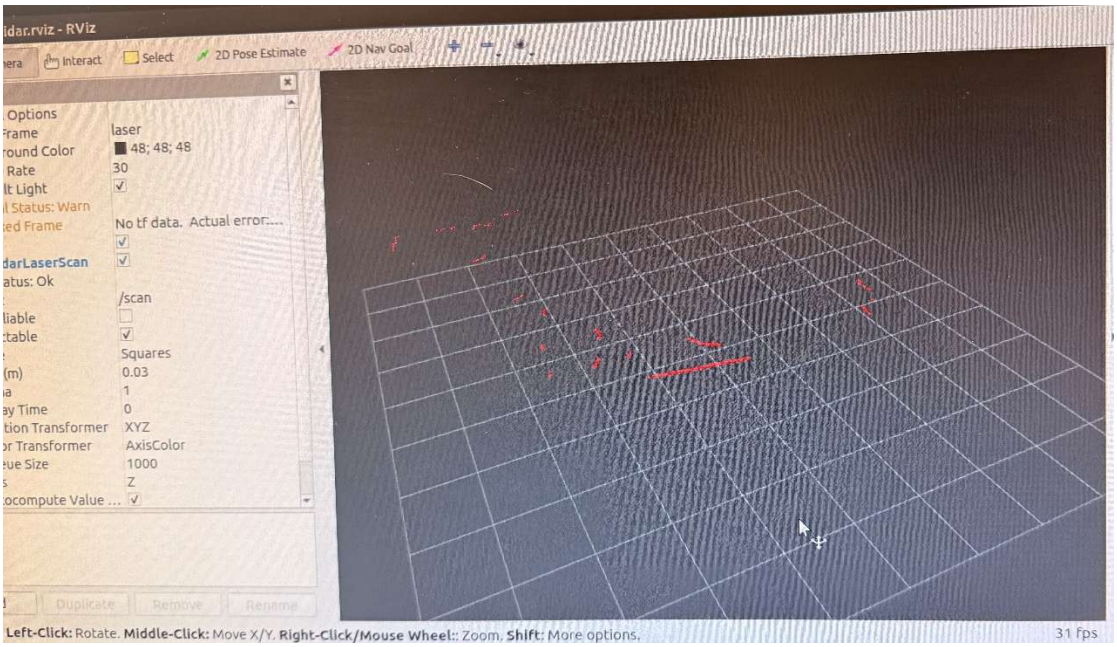
Question 5: Put an object such as a notebook or a cardboard at 270 degree around the lidar (at least 20 cm away), move the object back and forth and explain how and why the numbers change. Add screenshots of the array ranges when the object is the closest to the car and when it is the furthest from the car





In this experiment, we placed an object at the 270° position and observed how the Lidar's ranges array responded to physical movement. The data points in this array correspond to 0.5° increments in a counter-clockwise direction, starting from 0° at the front of the car near the VESC. When the notebook was moved very close to the sensor, the terminal displayed inf, indicating the object had entered the sensor's "dead zone" where distance cannot be calculated. As we moved the object further away, the values increased, representing the distance in meters. This confirmed that the 720 samples provide a full 360° spatial map, with specific indices like [360] representing the back of the car (180°), which is essential for the AEV's obstacle avoidance logic.

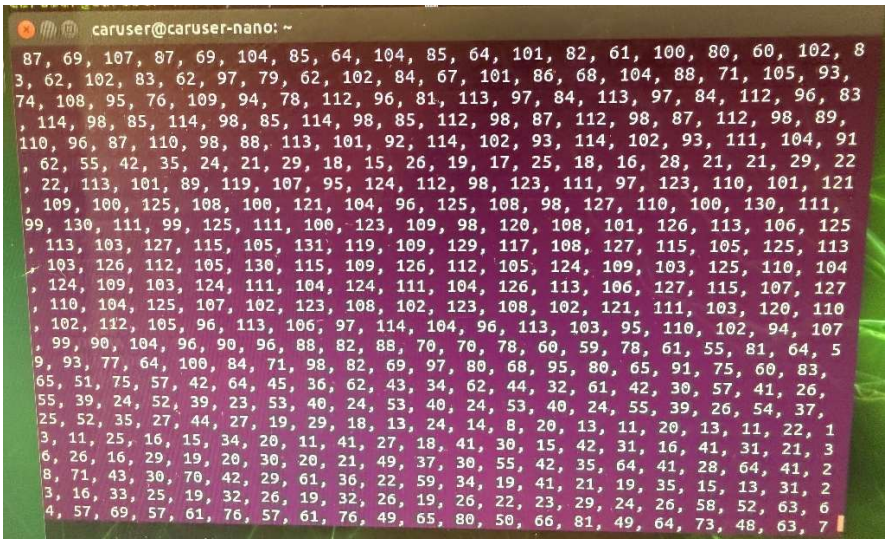
Question 6: Take a screenshot of the rviz page showing the point cloud map of the surrounding of the lidar and explain your observations of what the screenshot shows. Validate your observations by moving objects around the lidar and observe the change in the point cloud map



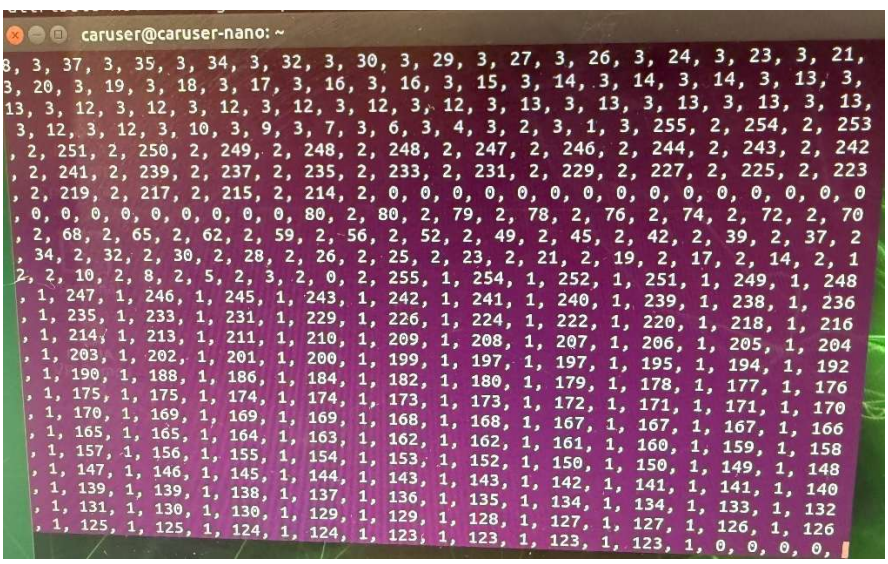
When we run the Lidar and go to rviz we can see that the visualized data represents our surrounding area. Image may not be as clear as we did not raise the vehicle to scan the entire room, but we did verify our observations by moving a notebook closer/further to the lidar scanner. The image shown above displays a longer line which is the back wall but started to break between the lines as it started scanning obstacles (monitors, people, suitcase) that was in the way and did not reflect and measure back.

Question 7: In your report, include a screenshot of the output with the 8-bit unsigned integer range from 0 to 255 as you echo /camera/color/image_raw and /camera/depth/image_rect_raw topics

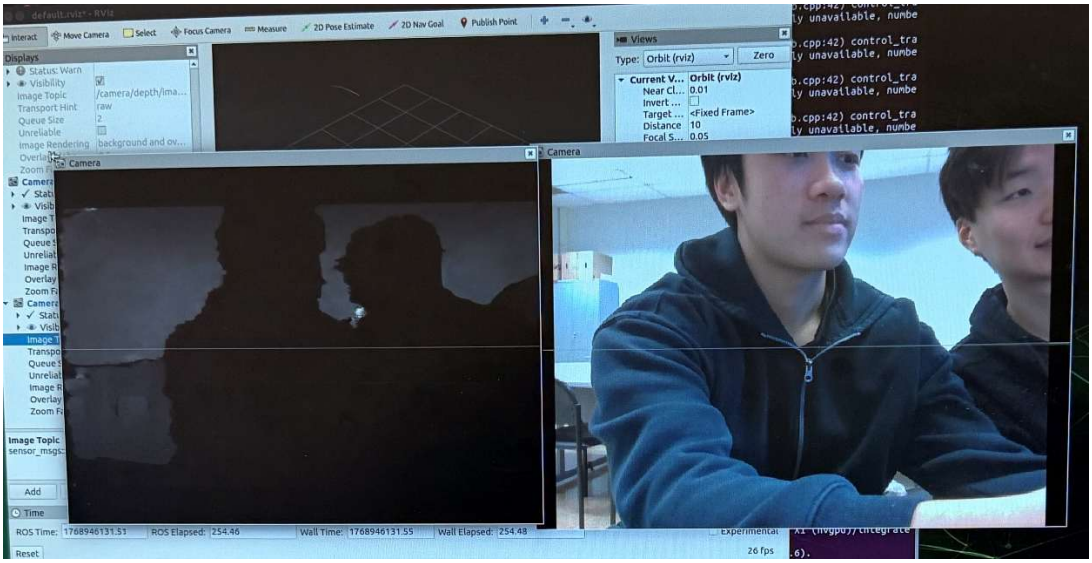
/camera/color/image_raw



/camera/depth/image_rect_raw



Question 8: Take a screenshot of the camera and depth camera simultaneously in one screenshot (Hint: you can do that by adding another camera page and selecting the topic you need). Explain briefly how the depth camera represents the raw coloured image. Also, take a screenshot of the raw coloured image showing ALL your team members



The depth camera represents the raw image by assigning a value to each pixel depicting the intensity of the color in the RGB image. The depth image essentially stores distance information from the camera to the object. By using a color gradient depth can be interpreted visually as an RGB image.

Question 9: The above explanation should have given you an idea about the parameters you added to the launch file. In your report, explain each line that you typed into the launch file (e.g., the four lines in the middle)

```
<node name="joy node" pkg="joy" type="joy node">
```

This line serves as the primary declaration for the joystick driver program, which is responsible for reading physical input data and publishing it as ROS messages to the rest of the system. The attribute name="joy node" assigns a unique identifier to the node within the active ROS system, while pkg="joy" specifies that the node belongs to the "joy" package, which contains the necessary joystick drivers. Finally, type="joy node" identifies the specific executable file that the system must run to initialize the controller interface.

```
<param name = "dev" value = "/dev/input/js0"/>
```

This line defines the primary runtime parameter used to configure the joy_node. It specifically identifies the device file associated with the joystick hardware. The value="/dev/input/js0" attribute points to the exact physical location of the controller within the Linux filesystem, which is the standard default path for the first joystick detected by the system.

```
<param name = "autorepeat rate" value = "2" />
```

This line defines the second parameter, which determines the frequency in Hertz at which the joy_node publishes messages. By setting the value="2", the auto-repeat rate is established at 2

Hertz, meaning the node will continue to publish two sets of messages every second even if there are no changes to the joystick input.

```
<param name = "coalesce interval" value = "0.05" />
```

This parameter defines the specific time delay in seconds between the publication of successive joystick messages. By assigning a value of 0.05, the system is configured to transmit new messages every 50 milliseconds. This low-latency interval is critical for maintaining high responsiveness to real-time input changes from the controller.

The "dev" parameter identifies the specific hardware by pointing to its device file path in the Linux system. The "autorepeat_rate" maintains system awareness by ensuring the node publishes a message at a set frequency even when the joystick is idle. Finally, the "coalesce_interval" minimizes latency by controlling the delay between updates, ensuring the car responds immediately to your physical inputs.

Question 10: Find out which element in the two arrays, axes and buttons correspond to which button by trying them all and seeing how the numbers change. Add screenshots showing the two arrays when each button is pressed and tested.

No buttons pressed (Default):

Axes: [0, 0, 1, 0, 0, 1, 0, 0]

Button: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Button A:

Button: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Button B:

Button: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Button X:

Button: [0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0]

Button Y:

Button: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]

LB:

Button: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]

RB:

Button: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]

Back:

Button: [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0]

Start:

Button: [0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0]

Logitech:

Button: [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0]

LJB:

Button: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0]

RJB:

Button: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]

Right trigger:

Axes: [0, 0, 1, 0, 0, -1, 0, 0]

Left Trigger:

Axes: [0, 0, -1, 0, 0, 1, 0, 0]

DPAD UP:

Axes: [0, 0, 1, 0, 0, 1, 0, 1]

DPAD RIGHT:

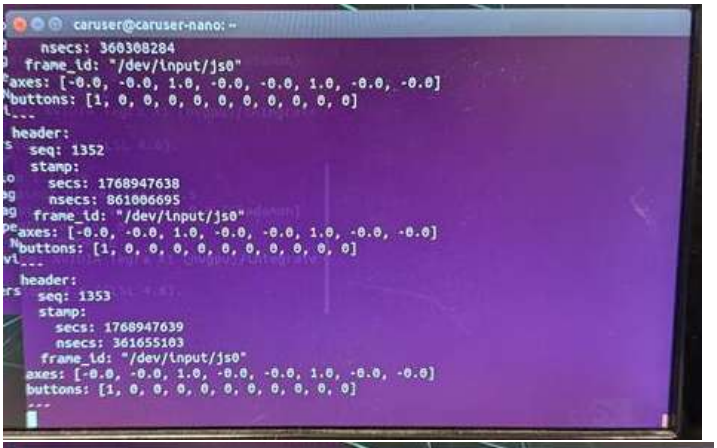
Axes: [0, 0, 1, 0, 0, 1, -1, 0]

DPAD DOWN

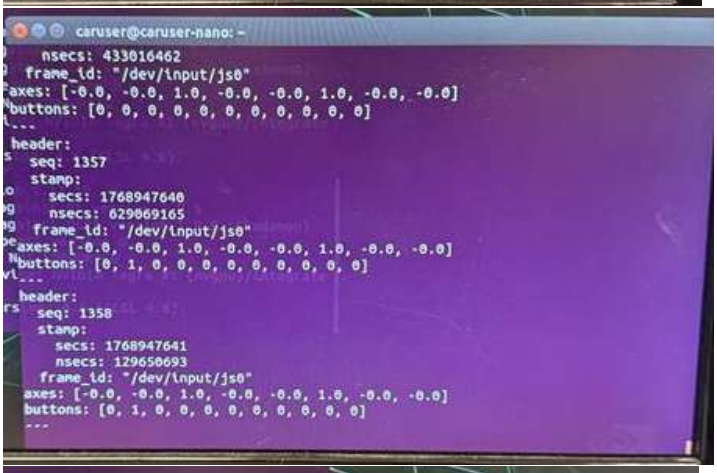
Axes: [0, 0, 1, 0, 0, 1, 0, -1]

DPAD LEFT

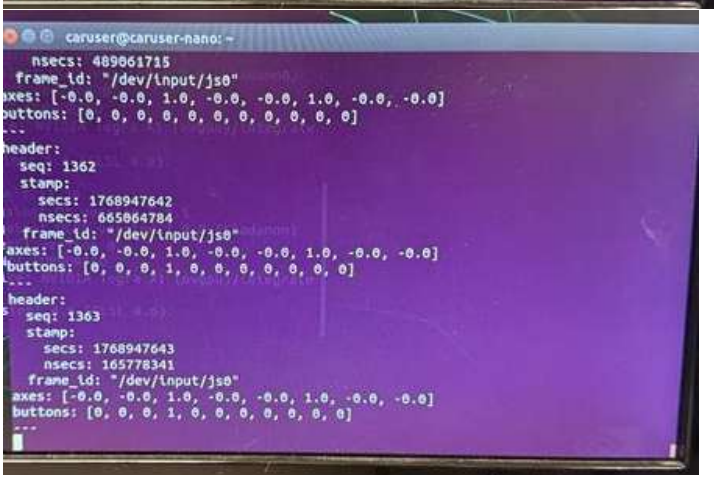
Axes: [0, 0, 1, 0, 0, 1, 1, 0]



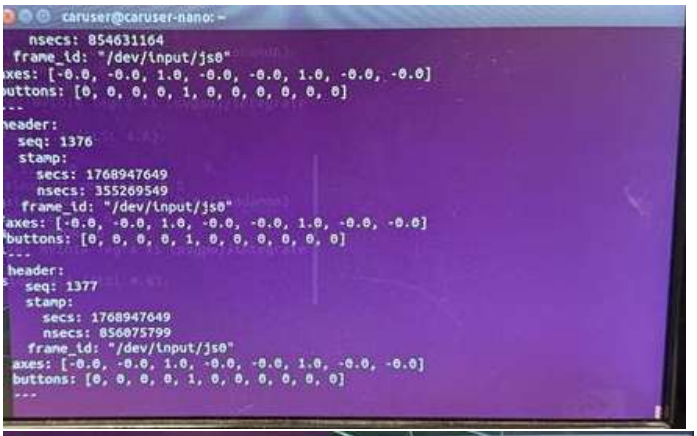
```
caruser@caruser-nano: ~
    nsecs: 360308284
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1352
  stamp:
    secs: 1768947638
    nsecs: 861006695
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1353
  stamp:
    secs: 1768947639
    nsecs: 361655103
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
```

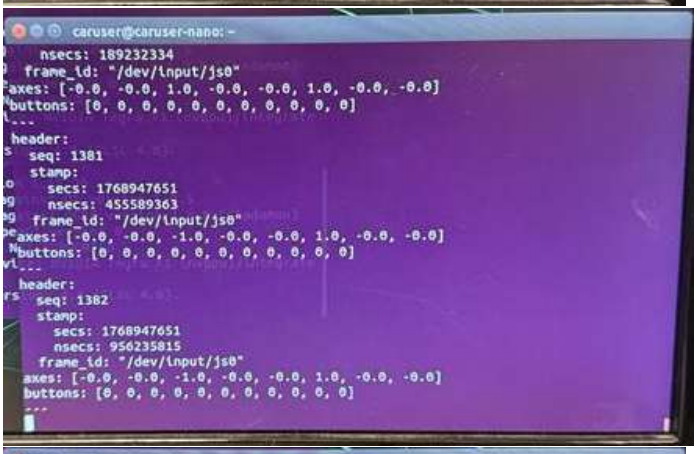


```
caruser@caruser-nano: ~
    nsecs: 433016462
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1357
  stamp:
    secs: 1768947640
    nsecs: 629069165
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1358
  stamp:
    secs: 1768947641
    nsecs: 129650693
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
```

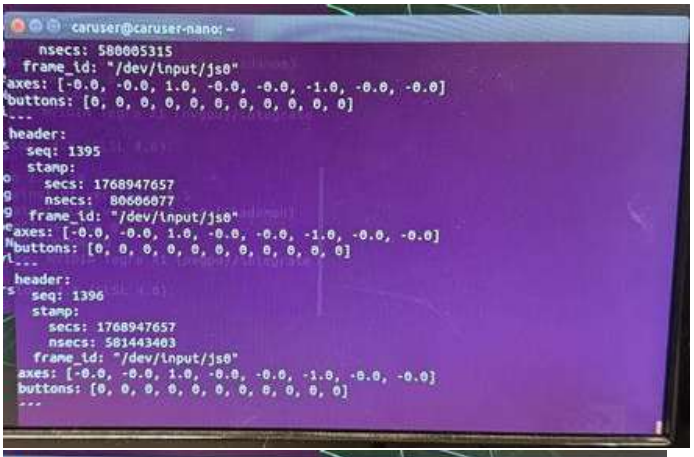


```
caruser@caruser-nano: ~
    nsecs: 489061715
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1362
  stamp:
    secs: 1768947642
    nsecs: 665064784
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1363
  stamp:
    secs: 1768947643
    nsecs: 165778341
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
---
```


caruser@caruser-nano: ~
nsecs: 854631164
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
---
header:
seq: 1376
stamp:
secs: 1768947649
nsecs: 355269549
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
---
header:
seq: 1377
stamp:
secs: 1768947649
nsecs: 856075799
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0]


caruser@caruser-nano: ~
nsecs: 189232334
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
seq: 1381
stamp:
secs: 1768947651
nsecs: 455589363
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, -1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
seq: 1382
stamp:
secs: 1768947651
nsecs: 956235815
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, -1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---

caruser@caruser-nano: ~
nsecs: 861291157
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
---
header:
seq: 1388
stamp:
secs: 1768947654
nsecs: 361886716
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]
---
header:
seq: 1389
stamp:
secs: 1768947654
nsecs: 862556598
frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0]



```
caruser@caruser-nano: ~
    nsecs: 580005315
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, -1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1395
  stamp:
    secs: 1768947657
    nsecs:  80606077
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, -1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1396
  stamp:
    secs: 1768947657
    nsecs: 581443403
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, -1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
```

```
caruser@caruser-nano: ~
    nsecs:  87420647
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, 1.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1401
  stamp:
    secs: 1768947659
    nsecs: 588253646
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, 1.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1402
  stamp:
    secs: 1768947660
    nsecs:  88903932
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, 1.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
```

```
caruser@caruser-nano: ~
    nsecs: 319484551
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1406
  stamp:
    secs: 1768947661
    nsecs: 459448676
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -1.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1407
  stamp:
    secs: 1768947661
    nsecs: 960453340
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -1.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
```

```
caruser@caruser-nano: ~
    nsecs: 961733118
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -1.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1410
  stamp:
    secs: 1768947663
    nsecs: 447518546
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1411
  stamp:
    secs: 1768947663
    nsecs: 671638137
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -1.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
```

```
caruser@caruser-nano: ~
    nsecs:  51561830
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1415
  stamp:
    secs: 1768947665
    nsecs: 175565927
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, 1.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1416
  stamp:
    secs: 1768947665
    nsecs: 676163803
  frame_id: "/dev/input/js0"
axes: [-0.0, -0.0, 1.0, -0.0, -0.0, 1.0, 1.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
```

```
caruser@caruser-nano: ~
    nsecs: 282615352
  frame_id: "/dev/input/js0"
axes: [-0.047212135046720505, 1.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1454
  stamp:
    secs: 1768947675
    nsecs: 783217528
  frame_id: "/dev/input/js0"
axes: [-0.047212135046720505, 1.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
header:
  seq: 1455
  stamp:
    secs: 1768947676
    nsecs: 283491781
  frame_id: "/dev/input/js0"
axes: [-0.047212135046720505, 1.0, 1.0, -0.0, -0.0, 1.0, -0.0, -0.0]
buttons: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
---
```